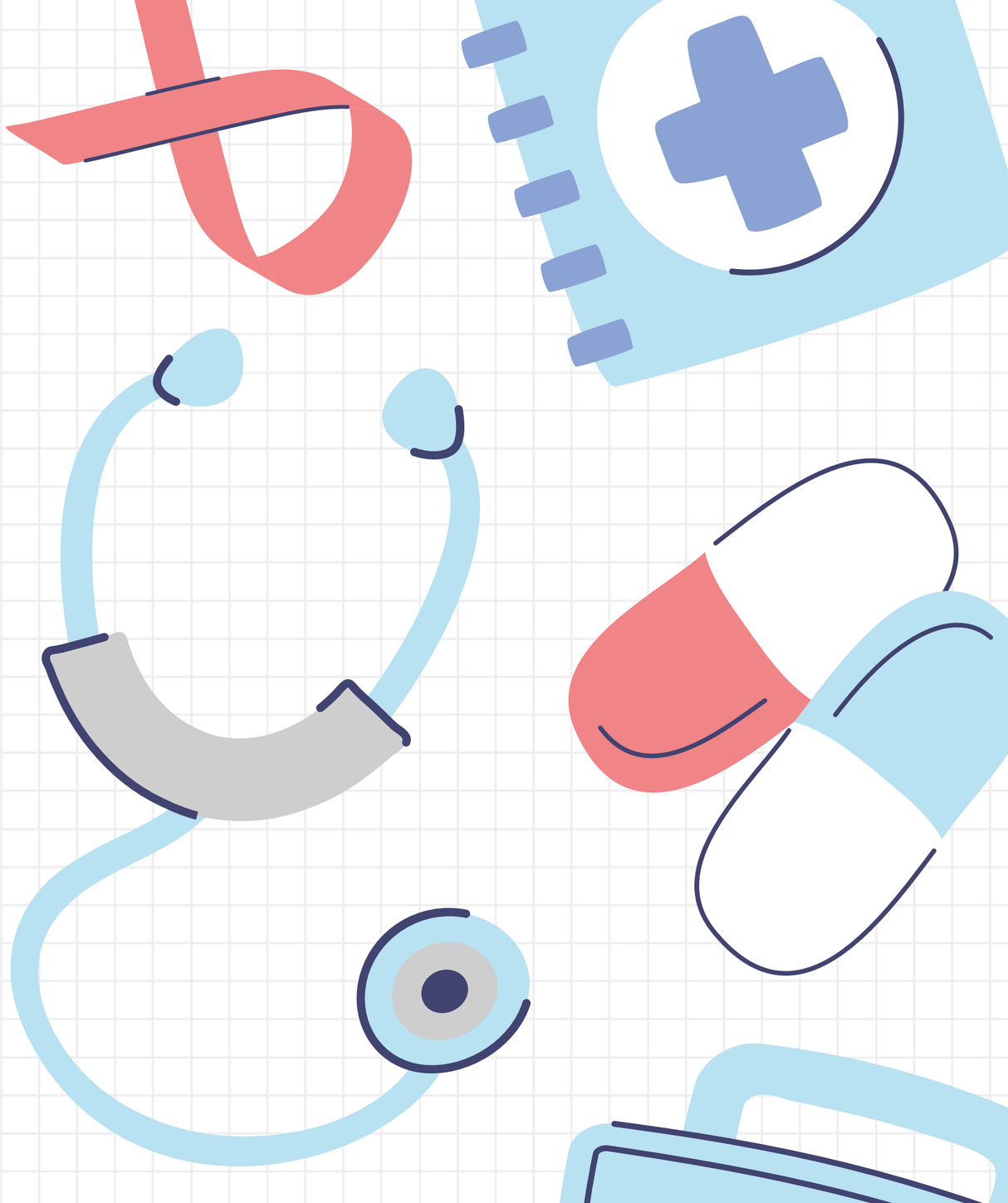
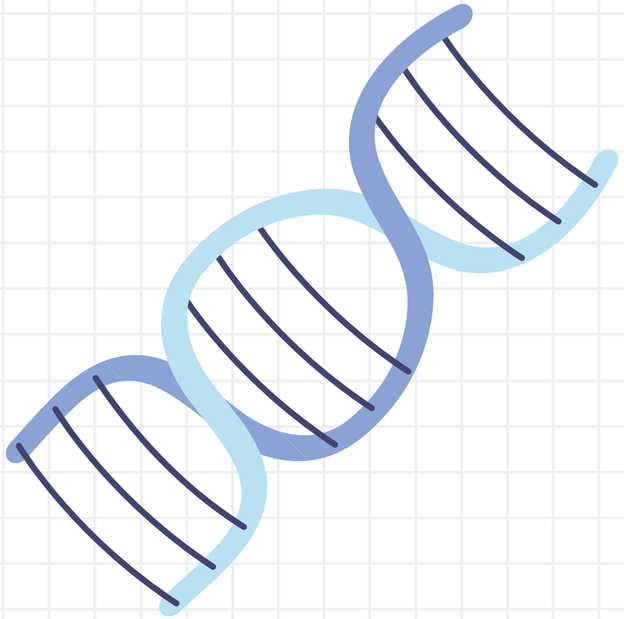
24JE0690,24JE0683,24JE0673

Group presentation

Similar Patient Case Retrieval





Content

1. Introduction
2. Concepts
3. Results

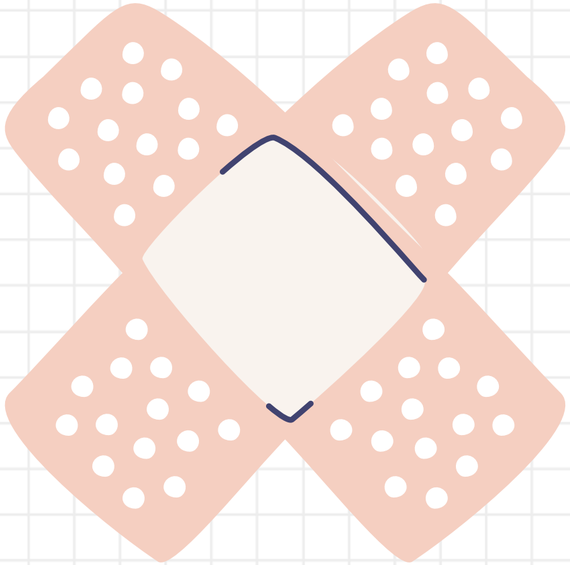
Introduction

We created:

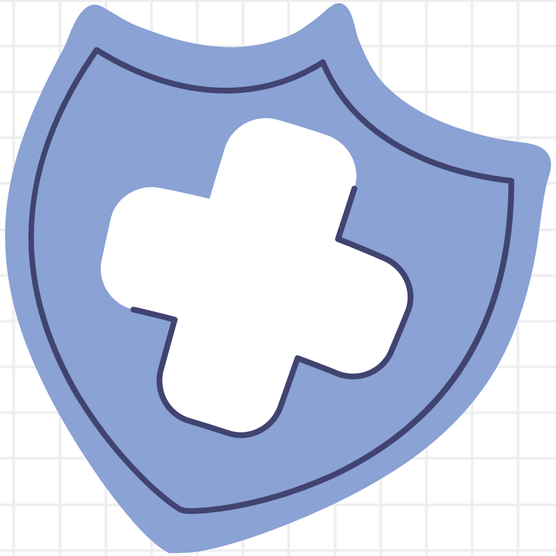
- A system that finds patients with similar clinical profiles using pure DBMS concepts (MongoDB Aggregation Pipelines).
- NO machine learning libraries used.
- Algorithm: Weighted Euclidean Distance computed entirely inside MongoDB.

Tech Stack

Language	Python 3.13
UI Framework	Streamlit 1.55
UI Navigation	streamlit-option-menu
Charts	Matplotlib
Database	MongoDB (Atlas or local)
DB Driver	PyMongo



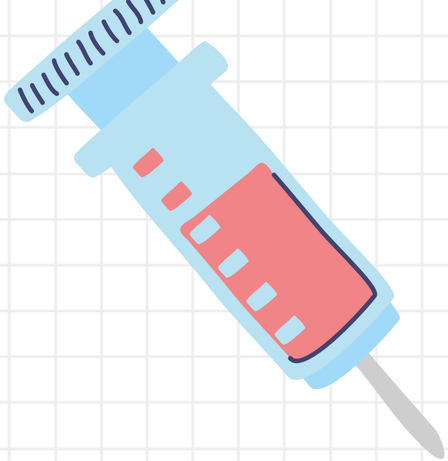
Concepts Covered

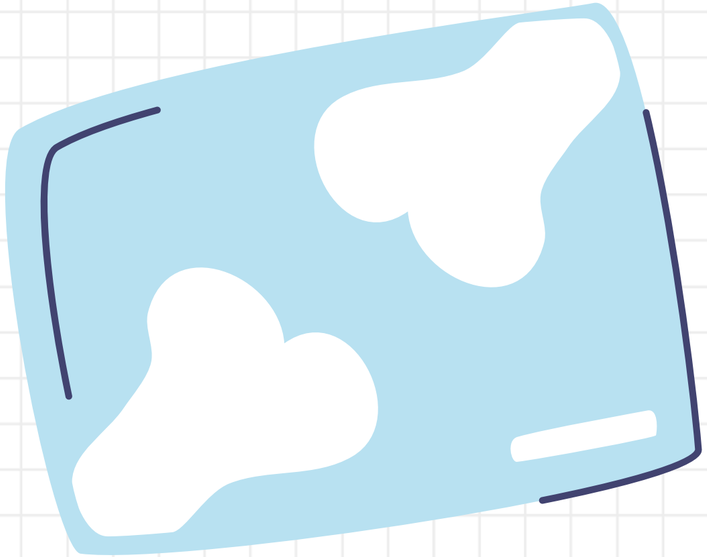


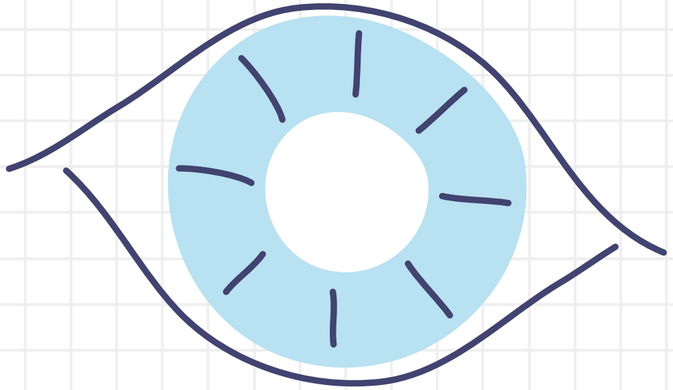
1. CRUD Operations

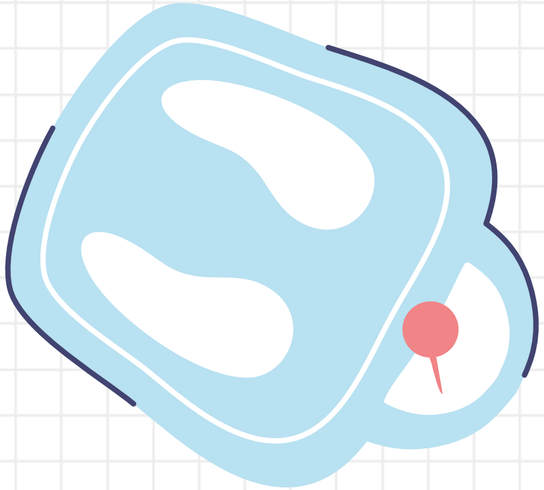
2. Joins & Aggregates

3. Indexes & Views



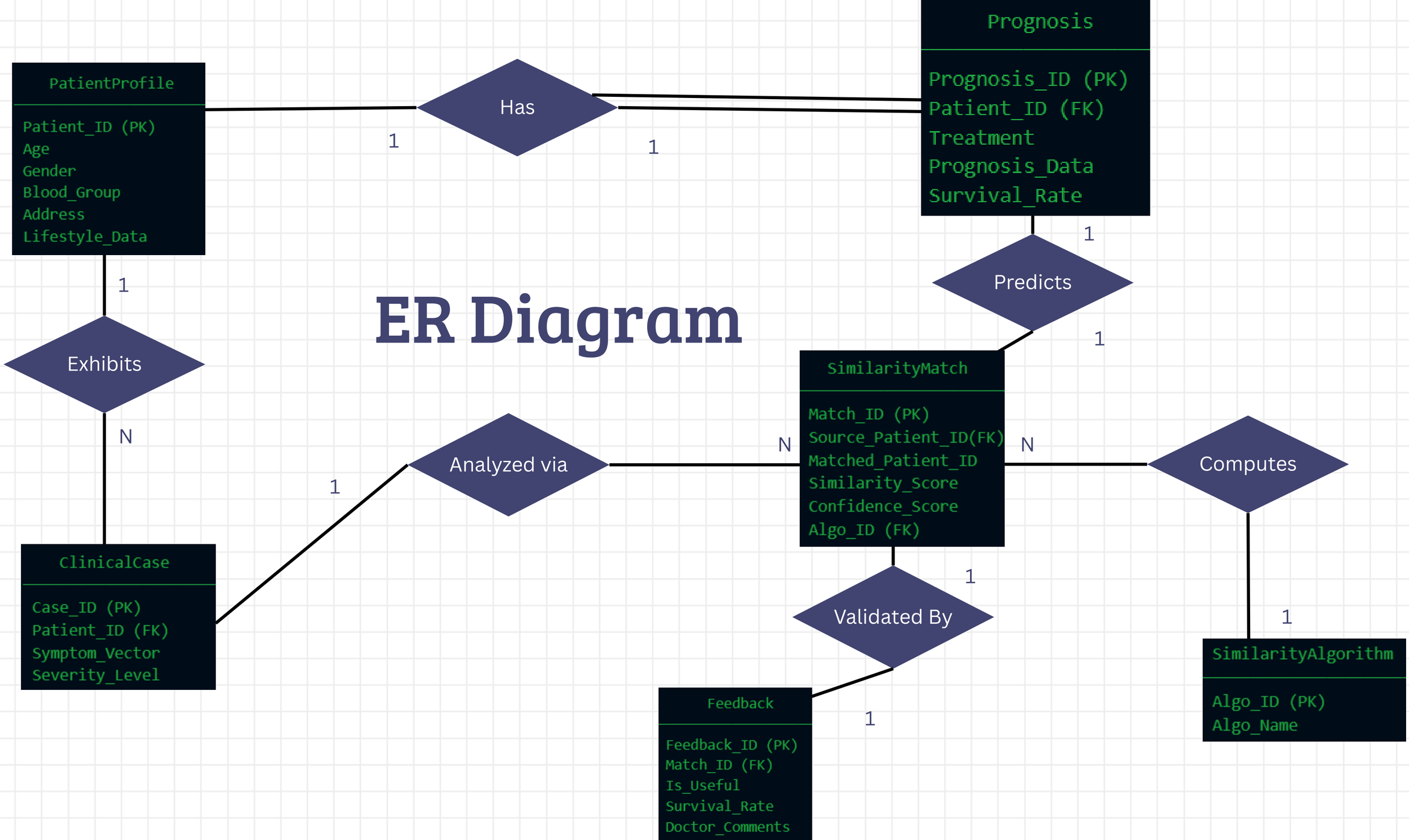






SQL Concept	MongoDB Implementation	File
`JOIN`	`$lookup`	`service.py`
`WHERE`	`$match`	`service.py`
`ORDER BY`	`$sort`	`service.py`
`LIMIT`	`$limit`	`service.py`
`SELECT`	`$project`	`service.py`
`AVG/MIN/MAX`	`$group`	`service.py`
`AFTER INSERT trigger`	post-insert `update_one()`	`service.py:submit_feedback()`
`CREATE VIEW`	multi- `$lookup` pipeline	`service.py:get_dashboard_view()`
`STORED PROCEDURE`	`find_similar_patients()` function	`service.py`
`CREATE INDEX`	`create_index()`	`collections.py`

ER Diagram


PatientProfile
Patient_ID (PK)
Age
Gender
Blood_Group
Address
Lifestyle_Data
Has
1
1
Prognosis
Prognosis_ID (PK)
Patient_ID (FK)
Treatment
Prognosis_Data
Survival_Rate
1
Predicts
1
1
Exhibits
N
ClinicalCase
Case_ID (PK)
Patient_ID (FK)
Symptom_Vector
Severity_Level
1
Analyzed via
N
SimilarityMatch
Match_ID (PK)
Source_Patient_ID(FK)
Matched_Patient_ID
Similarity_Score
Confidence_Score
Algo_ID (FK)
N
Computes
1
SimilarityAlgorithm
Algo_ID (PK)
Algo_Name
1
Validated By
1
Feedback
Feedback_ID (PK)
Match_ID (FK)
Is_Useful
Survival_Rate
Doctor_Comments

Algorithm

1. Min-Max Normalization Formula

Normalization scales numerical values to a standard range between 0.0 and 1.0 so that larger numbers (like age) don't overpower smaller numbers (like severity).

The general formula used is:

$$NormalizedValue = \frac{Value - MinValue}{MaxValue - MinValue}$$

Your exact implementations:

- Age Normalization:

$$Age_{norm} = \frac{PatientAge - DB_MinAge}{DB_MaxAge - DB_MinAge}$$

- Severity Normalization: (Since Severity is always on a scale of 1 to 10)

$$Severity_{norm} = \frac{PatientSeverity - 1}{9.0}$$

Algorithm

2. Similarity Match Formula (Weighted Euclidean Distance)

The core algorithm calculates the **Weighted Euclidean Distance** across the normalized fields to determine how far away two patients are from each other clinically.

Step A: Distance Calculation

$$Distance = \sqrt{w_{age}(SourceAge_{norm} - Age_{norm})^2 + w_{sev}(SourceSev_{norm} - Severity_{norm})^2 + w_{gen}(GenderDiff)^2 +}$$

(Where w represents the weights assigned to each feature from the frontend sliders).

Step B: Final Similarity Score Calculation

Because "Distance" means lower is better, the system converts distance into a **Similarity Score (from 0 to 1)**, where a score closer to `1.0` means a stronger match. The exact formula is:

$$SimilarityScore = \frac{1}{1 + Distance}$$

This is all executed purely via MongoDB Aggregation Pipeline mathematical operators (`$sqrt`, `$pow`, `$subtract`, `$add`, `$divide`).

Pipeline

Collection	Role	Key Fields
`patient_profiles`	Core demographics	`patient_id`, age, gender, blood_group
`clinical_cases`	Symptom/severity data	`case_id`, `patient_id`, severity_level (1-10), symptom_vector
`prognosis`	Treatment outcomes	`patient_id`, treatment, survival_rate
`similarity_algorithms`	Algorithm registry	`algo_id`, algo_name
`similarity_matches`	Computed results	source/matched patient IDs, similarity_score, confidence_score
`feedback`	Doctor validation	`match_id`, is_useful, doctor_comments

UI (Streamlit) → init_db() → seed (20) patients into 6 collections

↓

Doctor picks patient + weights → find_similar_patients() → 7-stage aggregation pipeline

↓

Top 10 results → save_matches() → INSERT to similarity_matches

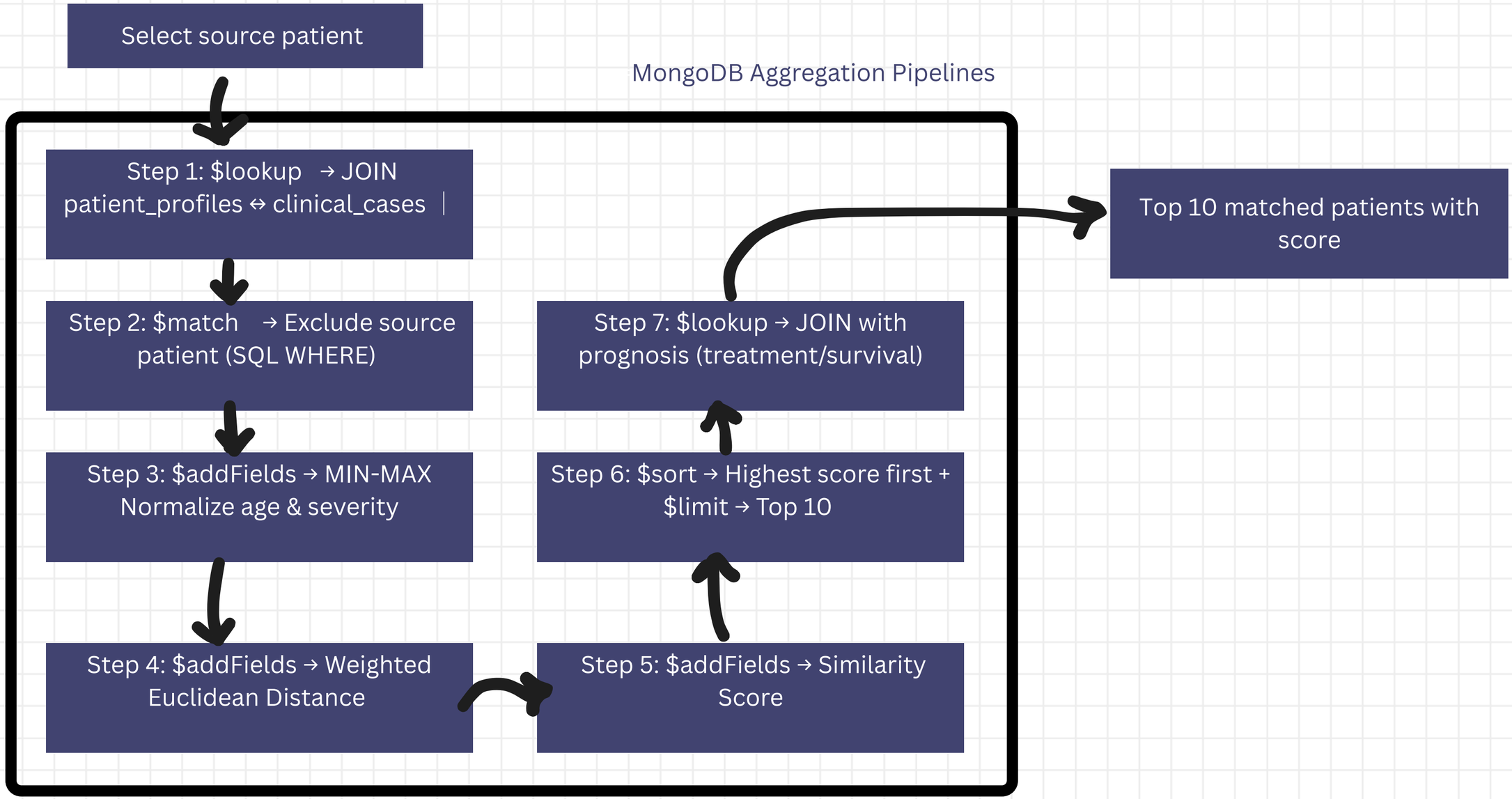
↓

Doctor validates → submit_feedback() → INSERT + auto-trigger confidence update

↓

get_dashboard_view() → 4-way JOIN → charts (bar, pie, horizontal bar)

Pipeline


Select source patient
MongoDB Aggregation Pipelines
Step 1: $lookup → JOIN patient_profiles ↔ clinical_cases
Step 2: $match → Exclude source patient (SQL WHERE)
Step 3: $addFields → MIN-MAX Normalize age & severity
Step 4: $addFields → Weighted Euclidean Distance
Step 5: $addFields → Similarity Score
Step 6: $sort → Highest score first + $limit → Top 10
Step 7: $lookup → JOIN with prognosis (treatment/survival)
Top 10 matched patients with score

Post Query

Results returned

↓

save_matches() → INSERT into similarity_matches collection

↓

Doctor reviews results → submits feedback (match_id, is_useful, comments)

↓

submit_feedback() → INSERT into feedback collection

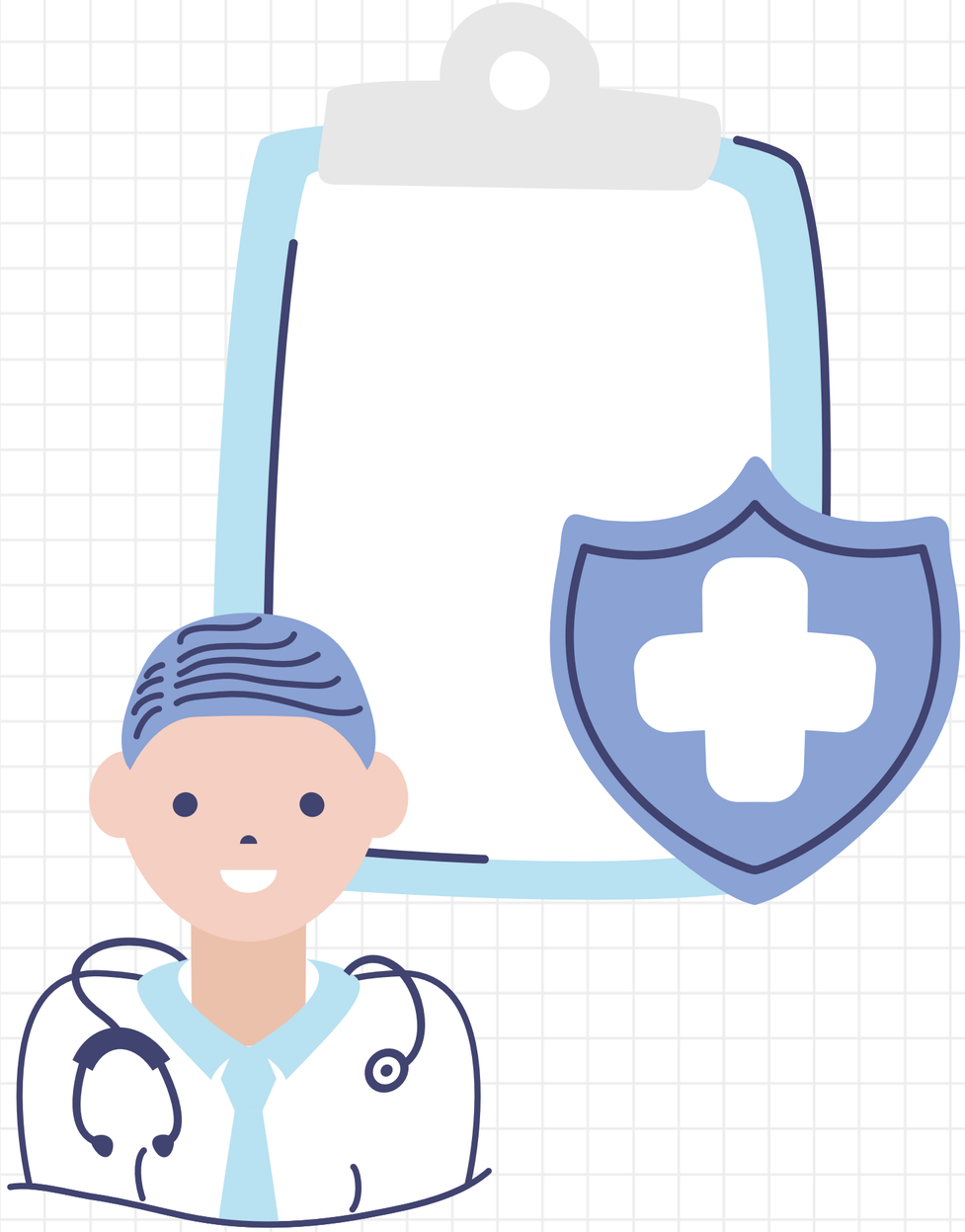
↓

[TRIGGER EQUIVALENT] Auto-recompute AVG(is_useful) via $group

→

update_one() → UPDATE similarity_matches.confidence_score

Results



View

Deploy

Category F > **Similar Patient Case Retrieval System**

🧬 Similar Patient Case Retrieval System

Module 36 — Pure DBMS-based patient similarity matching

🏠 Home | 🔗 ER Diagram | 📋 Tables | 🔍 Similarity Query | ⚡ Triggers & Views | 📊 Output

---

Similar Patient Case Retrieval System helps you find anonymized patient records with similar clinical profiles so you can understand typical treatments and outcomes.

👥 Patients	📋 Cases	🔗 Matches	💬 Feedback	💚 Avg Survival
20	20	10	0	78.9%

📥 Input Entities

1️⃣ **PatientProfile** — Demographics, lifestyle

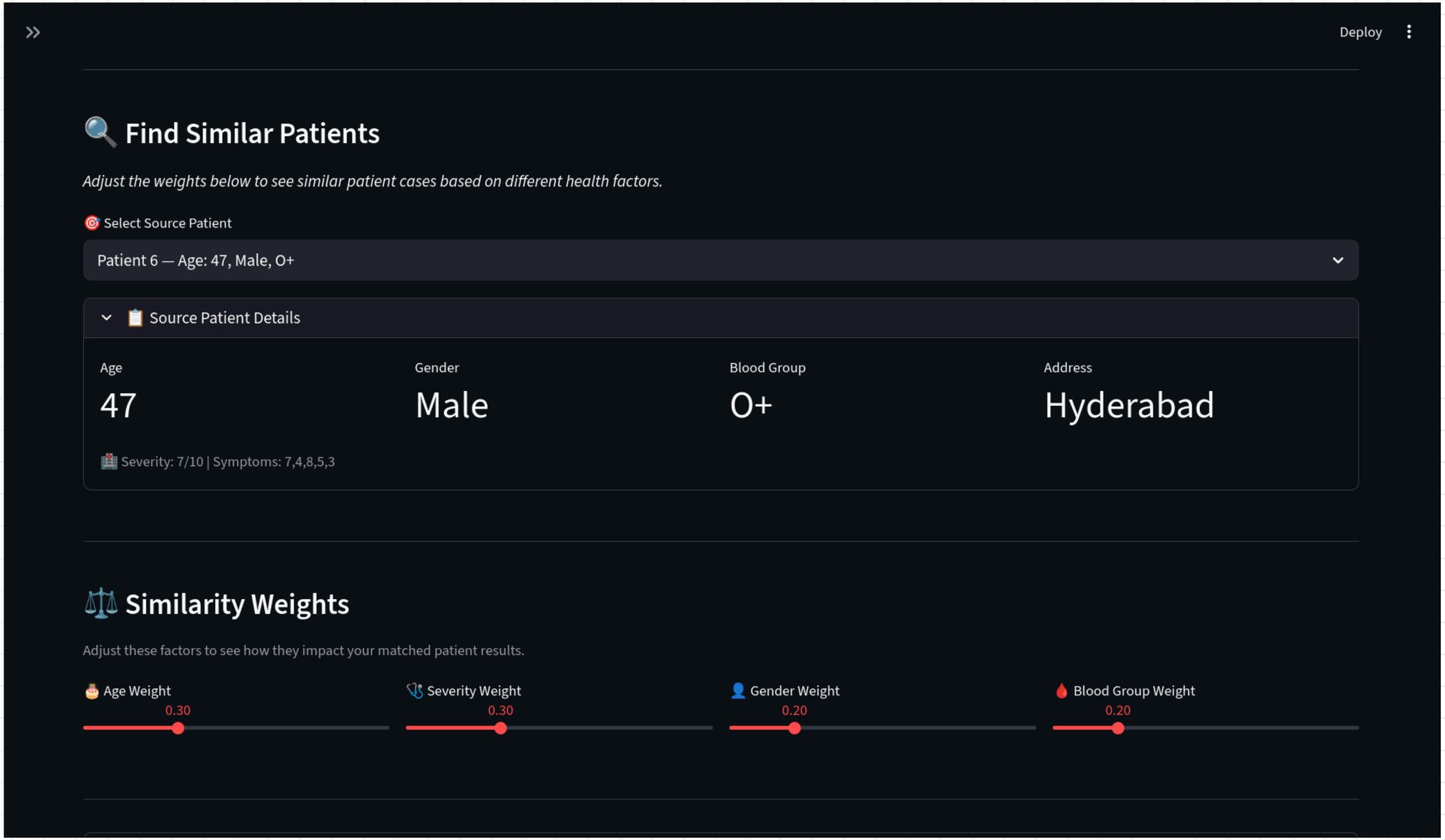
2️⃣ **ClinicalCase** — Symptoms, severity

📤 Output Entities

1️⃣ **SimilarityMatch** — Matched patients + scores

2️⃣ **Prognosis** — Treatment & survival data

Demonstration



Demonstration

🔧 MongoDB Aggregation Pipeline (View Code)

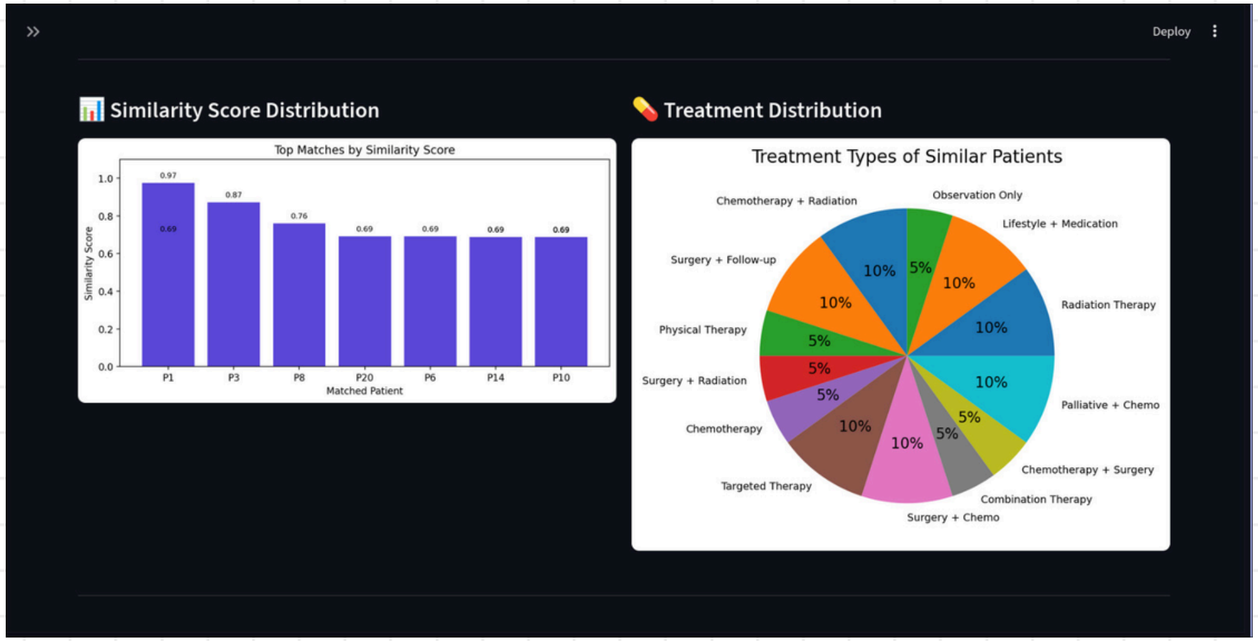
🚀 Find Similar Patients

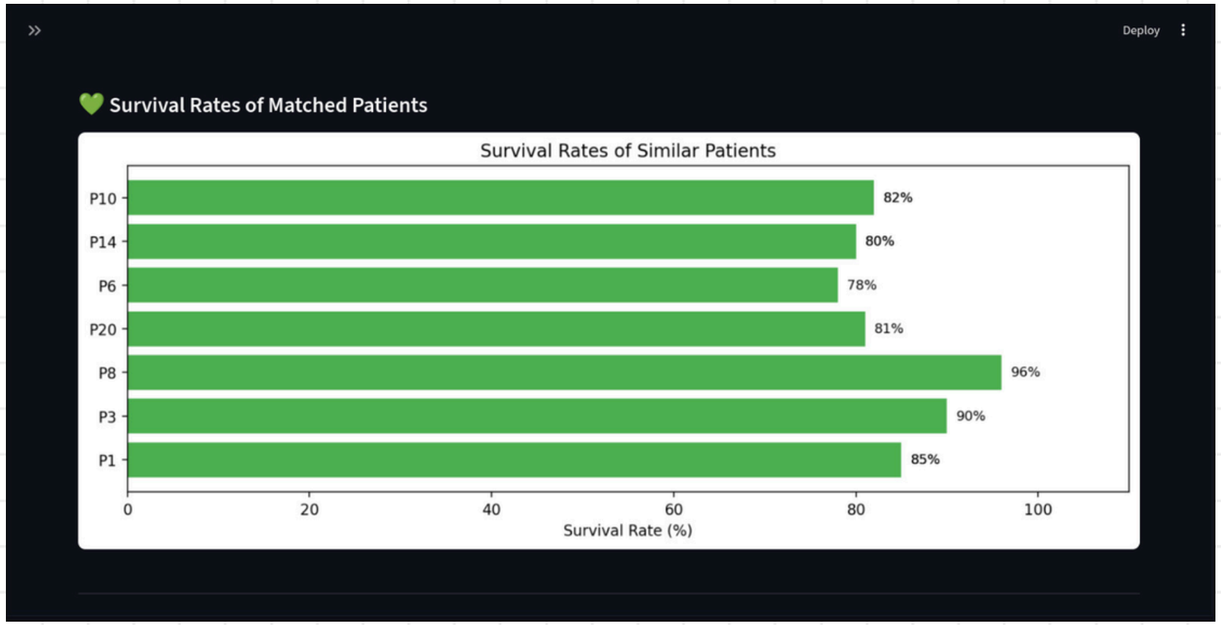
✅ Found 10 similar patients! (10 matches saved to DB)

📊 Top Similar Patients

Rank	Patient ID	Age	Gender	Blood Group	Severity	Similarity	Treatment	Survival %
1	1	45	Male	O+	7	0.9740	Chemotherapy + Radiation	85
2	20	48	Male	B+	7	0.6909	Surgery + Radiation	81
3	14	44	Male	A+	6	0.6882	Targeted Therapy	80
4	10	42	Male	A+	6	0.6867	Surgery + Chemo	82
5	19	57	Female	O+	7	0.6818	Combination Therapy	76
6	3	38	Male	B+	6	0.6816	Surgery + Follow-up	90
7	13	59	Female	O+	8	0.6762	Chemotherapy + Surgery	55
8	16	63	Male	O-	8	0.6669	Palliative + Chemo	42
9	12	36	Male	AB-	4	0.6645	Radiation Therapy	93
10	18	34	Male	A-	3	0.6502	Lifestyle + Medication	95

Demonstration





Queries Examples (ref. Repo)

```sql
CREATE TABLE PatientProfile (
    Patient_ID     INT PRIMARY KEY AUTO_INCREMENT,
    Age            INT NOT NULL,
    Gender         VARCHAR(10) NOT NULL CHECK (Gender IN ('Male', 'Female', 'Other')),
    Blood_Group    VARCHAR(5) NOT NULL,
    Address        VARCHAR(100),
    Lifestyle_Data VARCHAR(200)
);
```

```sql
CREATE INDEX idx_clinical_patient ON ClinicalCase(Patient_ID);
CREATE UNIQUE INDEX idx_prognosis_patient ON Prognosis(Patient_ID);
CREATE INDEX idx_match_score ON SimilarityMatch(Similarity_Score DESC);
CREATE INDEX idx_match_source ON SimilarityMatch(Source_Patient_ID);
CREATE INDEX idx_feedback_match ON Feedback(Match_ID);
```

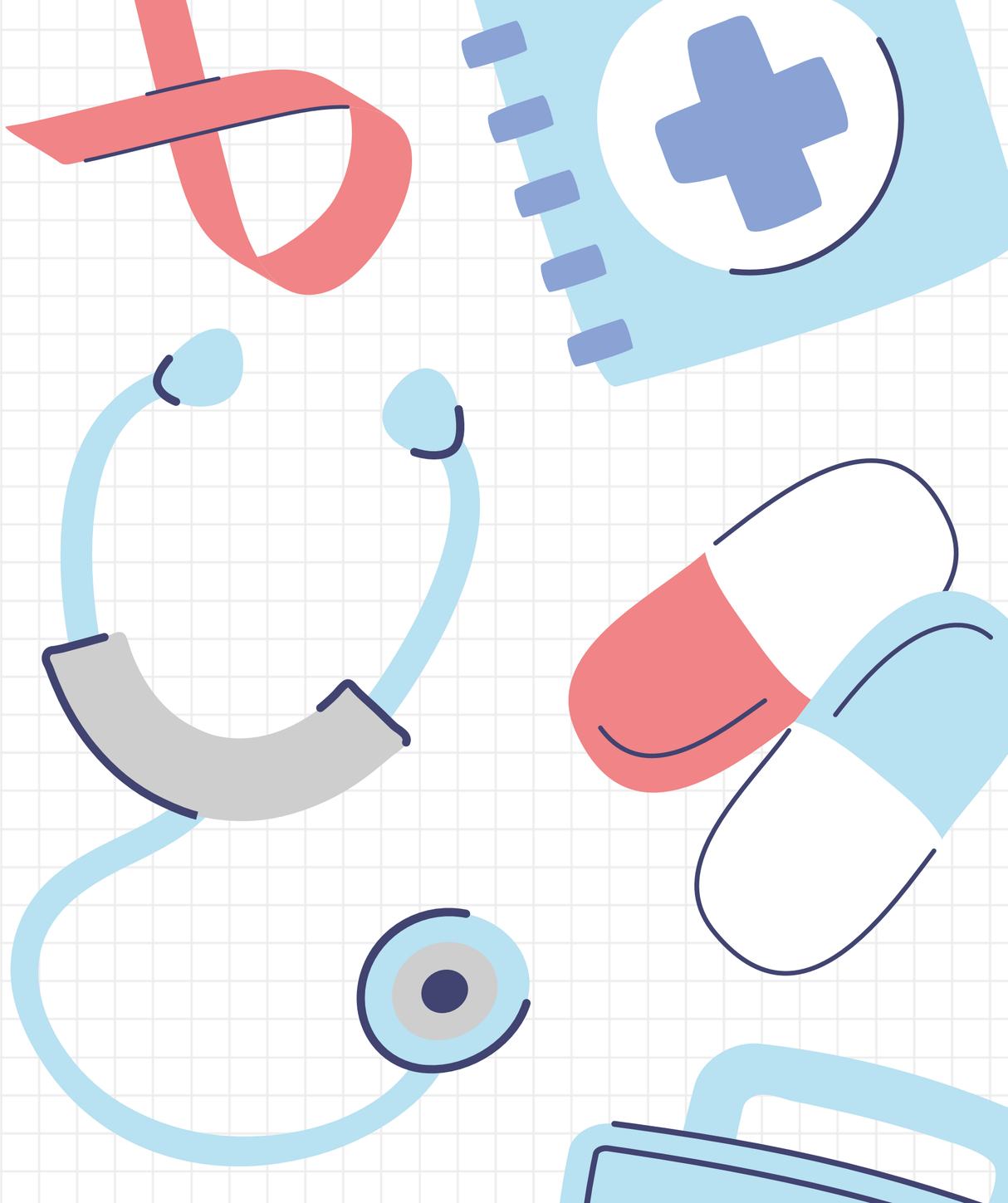
```sql
-- Get patient with clinical case and prognosis (JOIN)
SELECT pp.*, cc.Symptom_Vector, cc.Severity_Level, pr.Treatment, pr.Survival_Rate
FROM PatientProfile pp
JOIN ClinicalCase cc ON pp.Patient_ID = cc.Patient_ID
JOIN Prognosis pr ON pp.Patient_ID = pr.Patient_ID
WHERE pp.Patient_ID = 1;

-- Execute similarity search
CALL sp_find_similar_patients(1, 0.3, 0.3, 0.2, 0.2);

-- View dashboard
SELECT * FROM vw_patient_similarity_dashboard;

-- Get statistics (Aggregate Functions)
SELECT
    (SELECT COUNT(*) FROM PatientProfile) AS Total_Patients,
    (SELECT COUNT(*) FROM ClinicalCase) AS Total_Cases,
    (SELECT COUNT(*) FROM SimilarityMatch) AS Total_Matches,
    (SELECT COUNT(*) FROM Feedback) AS Total_Feedback,
    (SELECT ROUND(AVG(Survival_Rate), 1) FROM Prognosis) AS Avg_Survival_Rate;
```

24JE0690,24JE0683,24JE0673

Group presentation

Thank you!